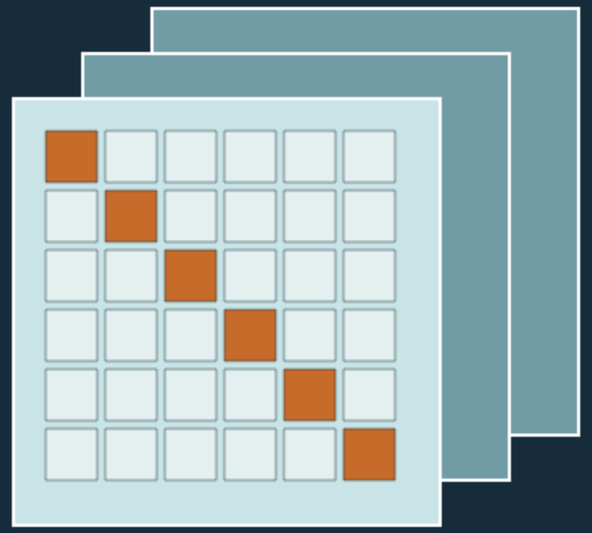
Convolutional neural networks

A tutorial in spatial structure, learned filters, and working code

Local connections and shared weights
Feature maps and spatial resolution
Training, evaluation, and prediction



Learned spatial filters

20 slides with detailed notes
Python + NumPy laboratory included

Images are arrays with spatial meaning

The tensor axes specify where values live and what each value measures.

One numerical channel

1.0	0.0	0.0	0.0	0.0	0.0
0.0	1.0	0.0	0.0	0.0	0.0
0.0	0.0	1.0	0.0	0.0	0.0
0.0	0.0	0.0	1.0	0.0	0.0
0.0	0.0	0.0	0.0	1.0	0.0
0.0	0.0	0.0	0.0	0.0	1.0

Illustrative 6 × 6 patch

X has shape [B, C, H, W]

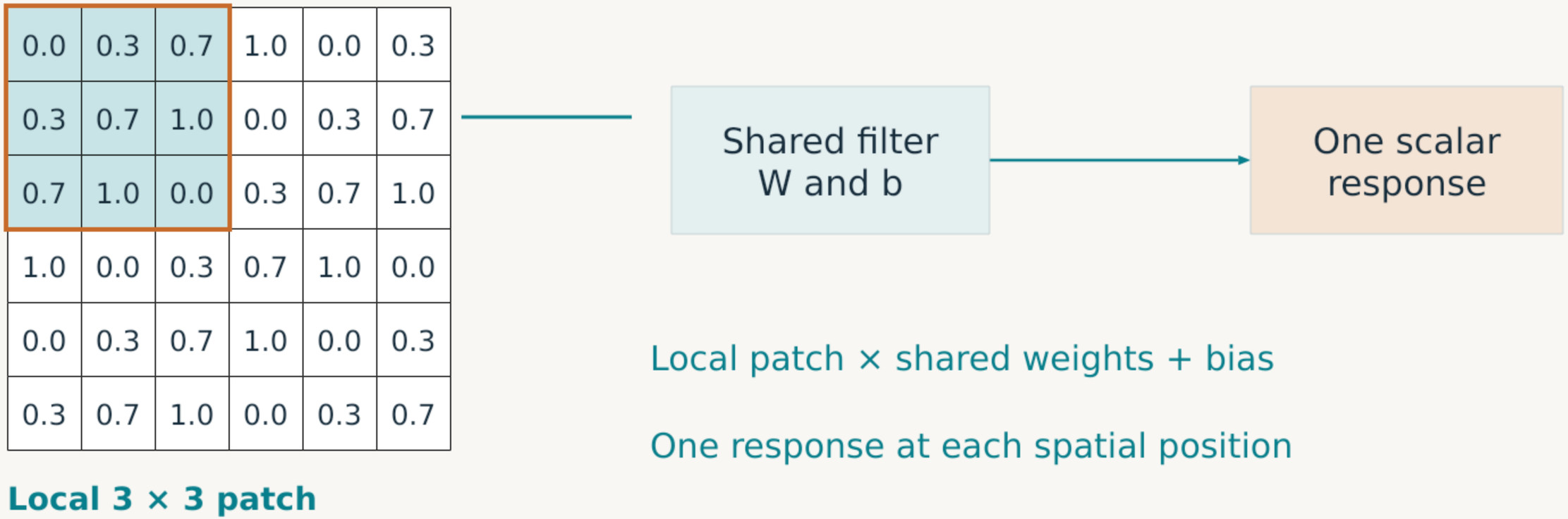
B: images in the batch
C: measurement channels
H, W: spatial coordinates

Lab input: [32, 1, 12, 12]

RGB images have 3 color channels.
Logical masks are separate measurements.

Local connections and shared filters

One small weight array scans the image and produces one feature map.



The filter parameters stay fixed during one forward pass. Training updates them afterward.

A convolution calculated by hand

Deep-learning libraries usually implement cross-correlation: the kernel is not flipped.

Input X

1	2	0
0	1	3
2	1	0

$*$

Kernel W

1	0
0	-1

$=$

Output, stride 1

0	-1
-1	1

Bias = 0. No padding. Output size = 2 × 2.

Top-left response: $1(1) + 2(0) + 0(0) + 1(-1) = 0$

Stride, padding, and output size

Kernel size sets the local footprint. Stride sets the distance between output samples.

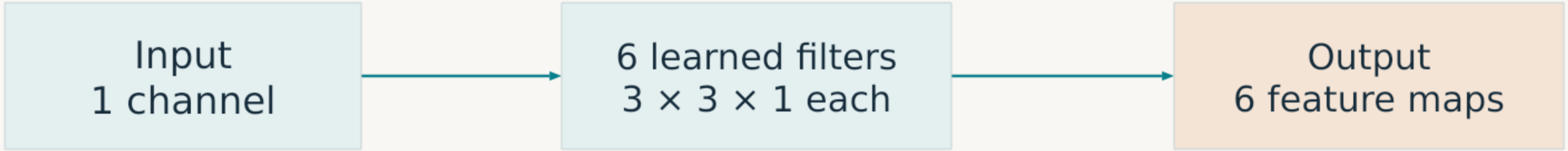
$$H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

Input H	Kernel K	Padding P	Stride S	Output H
12	3	0	1	10
12	3	1	1	12
12	3	1	2	6

Dilation 1 in this equation. For dilation D, replace K by $D(K-1)+1$.

Channels and trainable parameters

Each filter spans every input channel and has one shared bias.



W: [Cout, Cin, K, K]

Parameters = $C_{out}(C_{in} K^2 + 1)$

6 filters on 1 channel: $6(1 \times 9 + 1) = 60$

Output shape: [B, Cout, Hout, Wout]

Nonlinear feature maps

ReLU changes each response independently while preserving its spatial location.

Responses before ReLU

-2	1	3
0	-1	2
4	-3	1

ReLU

Activations after ReLU

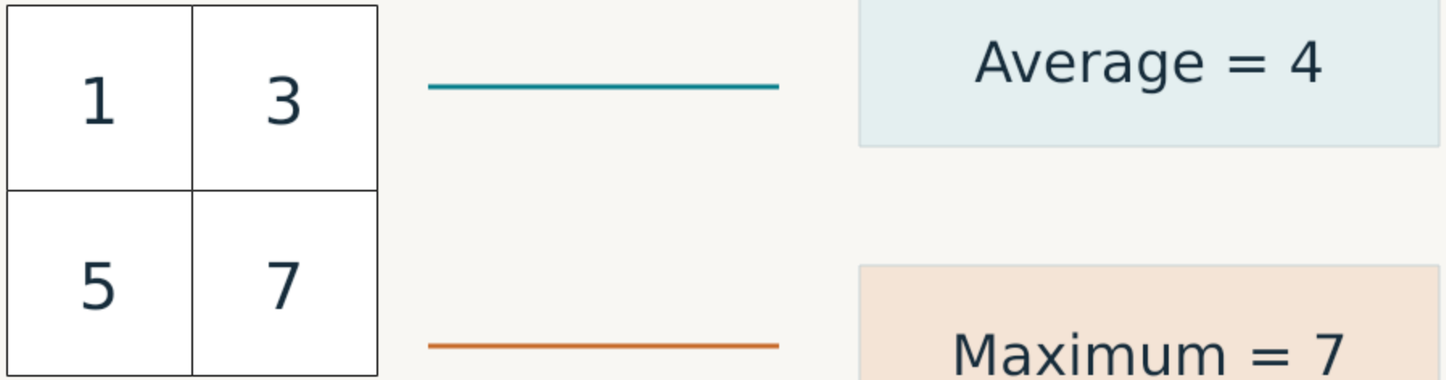
0	1	3
0	0	2
4	0	1

ReLU(z) = max(0, z). The final output layer remains linear.

Pooling summarizes nearby responses

Average pooling preserves a local mean. Max pooling preserves the largest response.

One 2 × 2 region



Average pooling:
(1+3+5+7) / 4

The lab uses average pooling with kernel 2 and stride 2.

Average-pool gradient: each input receives one quarter of the output gradient.

Receptive fields grow across layers

A later activation depends on a larger region of the original image.

$$r_{new} = r + (K-1)j \qquad j_{new} = jS$$

Stage	Kernel / stride	Receptive field r	Jump j
Input	—	1	1
Conv 1	3 / 1	3	1
Average pool	2 / 2	4	2
Conv 2	3 / 1	8	2

Global average pooling then summarizes all positions in each final feature map.

The complete laboratory CNN

Two local feature extractors feed a compact output head.

Stage	Output shape	Parameters
Input	B × 1 × 12 × 12	0
Conv 3×3 + ReLU	B × 6 × 12 × 12	60
Average pool 2×2	B × 6 × 6 × 6	0
Conv 3×3 + ReLU	B × 8 × 6 × 6	440
Global mean + 3-class head	B × 3	27

527 classification parameters. Replacing the head with one output gives 509.

Classification and regression heads

The feature extractor can serve different targets by changing the final head and loss.

Image classification

3 logits for 3 exclusive classes

$p_k = \exp(z_k) / \Sigma_j \exp(z_j)$
Loss = $-\log p_{true}$

Metrics: accuracy and macro-F1

Image regression

1 real-valued output

Loss = mean((prediction − target)²)
Report errors in original units.

Metrics: MAE, RMSE, and R^2

Image data and labels

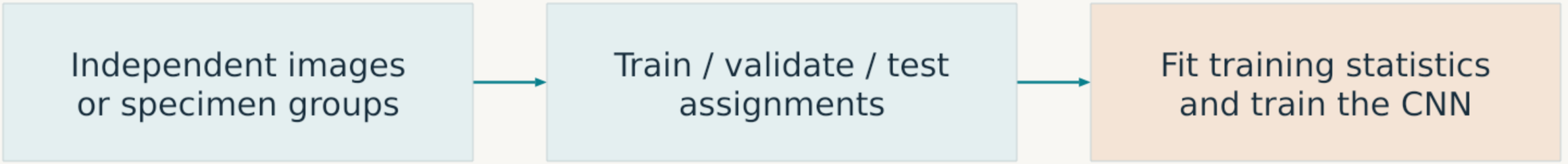
The row order of inputs, labels, and split assignments must agree.

Array / value	Representation	Important check
X	[N, C, H, W] floating point	Even H/W ≥ 4; fixed channel meaning
Classification y	[N], integer IDs 0…K−1	Every evaluated class occurs in training
Regression y	[N], finite real values	Units and target scaling recorded
Optional split	[N], IDs 0/1/2	Independent images or whole groups

Training-only channel statistics travel with the saved model.

Splits and augmentation

Related images belong together, and transformations must preserve the label meaning.



Training fits transformations and weights.
Validation selects settings and the best checkpoint.
The final test uses a frozen model.

Augmentation belongs inside training and must respect classification labels or regression values.

Backpropagation through shared weights

A filter gradient adds contributions from every image and every position where the filter was used.

$$\partial L/\partial W[o,c,u,v] = \Sigma b,i,j\ \delta[b,o,i,j]\ Xpatch[b,c,i,j,u,v]$$

$$\partial L/\partial b[o] = \Sigma b,i,j\ \delta[b,o,i,j]$$

Input gradients add where receptive fields overlap.

ReLU backward: multiply by $(z > 0)$.

Average-pool backward: divide across four inputs.

Global-mean backward: divide across all spatial positions.

The training loop and optimizer

Minibatches train the filters and output head together.

```
for ids in shuffled_batches:
    loss, grads = net.loss_grad(
        batch, labels, task)
    optimizer.step(net.params, grads)

val_loss = score(validation_data)
# Keep the lowest-validation-loss weights.
# Evaluate test data only after selection.
```

Batch size: 32
AdamW learning rate: 0.003
Weight decay: 0.0001
Gradient norm cap: 5
Maximum epochs: 80
Early-stop patience: 15

Full runnable source and tests accompany the presentation.

Convolution in executable NumPy

The implementation exposes image patches and the weighted sum.

```
xp = np.pad(X, padding)
patches = sliding_window_view(
    xp, (3, 3), axis=(2, 3))

Z = np.einsum(
    "bchwij,ocij->bohw", patches, W)
Z += b[None, :, None, None]
A = np.maximum(Z, 0)
```

Patch axes
b: batch
c: input channel
h,w: output position
i,j: kernel position

o: output channel

Full runnable source and tests accompany the presentation.

Training, testing, and saved-model prediction

One command runs the lab; separate commands make each experiment explicit.

```
python run_all.py

python lab.py train --data data/classification.npz \
    --out runs/experiment_2 --lr 0.001
python lab.py evaluate --data data/classification.npz \
    --run runs/experiment_2
```

Batch training

Forward → loss → backward
AdamW update → validation

Saved checkpoint

Weights and feature statistics
Split indices and dataset hash

Independent testing

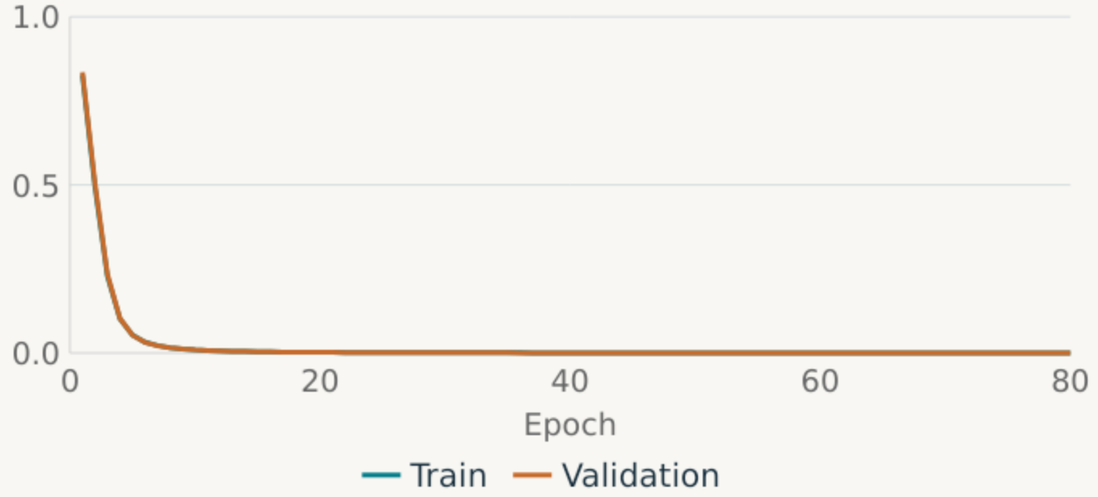
Restore selected weights.
Report held-out metrics.

The README documents the NPZ input contract and all command-line options.

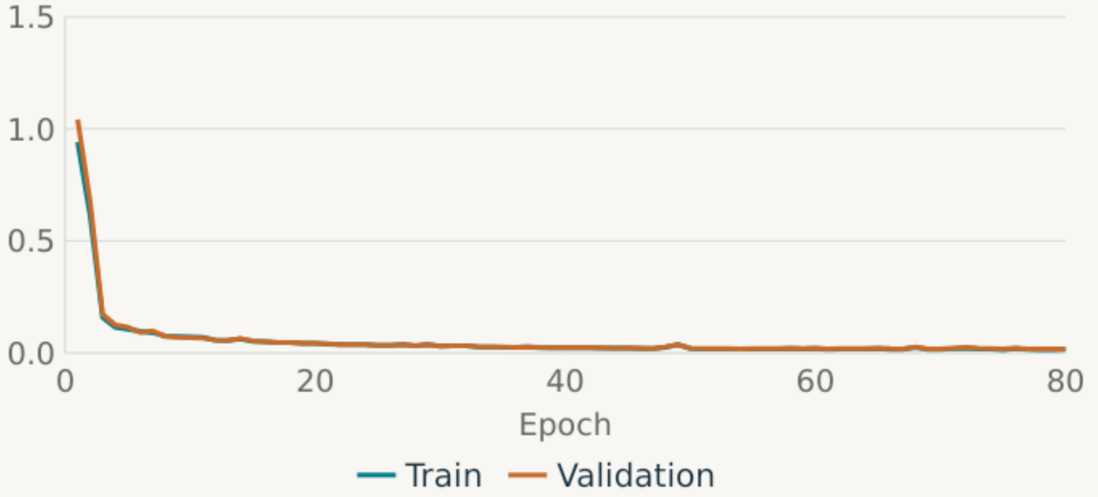
Measured results on held-out examples

Learning curves and metrics come from the included CPU reference runs.

Classification: cross-entropy



Regression: standardized MSE



Test accuracy 100.00%

Majority baseline: 34.43%
Macro-F1 1.0000
183 test examples; best epoch 80

Test RMSE 0.0361

Training-mean baseline: 0.2663
MAE 0.0275 R^2 0.9816
180 test examples; best epoch 78

Synthetic teaching tasks. These scores do not establish real-world performance.

Diagnostics and useful tests

Spatial structure creates failure modes that a single accuracy number can hide.

Observation	Possible cause	Targeted check
Poor fit even on a tiny batch	Shape, label, or gradient bug	Hand example and finite differences
Train improves, validation worsens	Overfitting or correlated split	Group split and earlier stopping
Errors after a small shift	Borders or downsampling sensitivity	Controlled translation tests
Amplitude regression fails	Brightness normalization removed signal	Inspect units and preprocessing

Seven tests passed in this lab. CPU implementation, no GPU backend.

Laboratory and further reading

The full source, reference runs, and detailed notes accompany these slides.

Exercises

1. Derive the 527-parameter count.
2. Implement max pooling and its backward pass.
3. Compare filters and learning rates on validation.
4. Test shifts, rotations, and brightness changes.

Primary references

Stanford CS231n convolution notes
Deep Learning, Chapter 9
AdamW paper
scikit-learn leakage guidance

Companion lab: source code, automated tests, datasets, reference checkpoints, training histories, test metrics, prediction examples, and complete lecture notes.